

PostgreSQL Database Assignment & Study Notes

1. What is PostgreSQL?

PostgreSQL is a powerful, open-source relational database management system used to store, manage and retrieve application data. It supports standard SQL plus advanced features such as JSONB, CTEs, rich data types, indexes and transactions.

2. Important Terms

Term	Simple Meaning
Database	A collection of related data.
Schema	A namespace used to organize database objects. The default schema is public.
Table	Stores information in rows and columns.
Row	One complete record.
Column	One type of information.
Primary Key	A unique identifier for each row.
Foreign Key	Connects related tables.
Constraint	A rule that protects data quality.
Query	An SQL command.
Index	Helps PostgreSQL find data faster.
View	A saved query that acts like a virtual table.

3. Database Relationships

```
categories  1 ---- many  products
customers   1 ---- many  orders
orders      1 ---- many  order_items
products    1 ---- many  order_items
```

Explanation: A category can contain many products. A customer can have many orders. An order can contain many products through order_items.

4. Create and Connect to the Database

```
CREATE DATABASE restaurant_assignment_db;
```

Explanation: Run this while connected to the default postgres database. CREATE DATABASE creates a new PostgreSQL database.

In SQL Shell (psql), connect to it with:

```
\c restaurant_assignment_db
```

Explanation: \c is a psql command, not a normal SQL statement. It changes your current connection to the selected database.

5. Create Tables

5.1 Categories

```
CREATE TABLE categories (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Explanation: IDENTITY automatically generates IDs. PRIMARY KEY makes each ID unique. UNIQUE prevents duplicate category names.

5.2 Products

```
CREATE TABLE products (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  category_id INTEGER NOT NULL,  
  name VARCHAR(150) NOT NULL,  
  description TEXT,  
  price NUMERIC(10,2) NOT NULL CHECK (price >= 0),  
  is_active BOOLEAN DEFAULT TRUE,  
  metadata JSONB DEFAULT '{}'::jsonb,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_product_category  
    FOREIGN KEY (category_id) REFERENCES categories(id)  
);
```

Explanation: NUMERIC is suitable for money. CHECK prevents negative prices. BOOLEAN stores true/false. JSONB stores flexible JSON information. The foreign key links each product to a category.

5.3 Customers

```
CREATE TABLE customers (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name VARCHAR(120) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  phone VARCHAR(30),  
  city VARCHAR(100),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Explanation: UNIQUE on email prevents duplicate customer email addresses.

5.4 Orders

```
CREATE TABLE orders (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  customer_id INTEGER NOT NULL REFERENCES customers(id),  
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(30) DEFAULT 'Pending',  
  total_amount NUMERIC(10,2) DEFAULT 0.00 CHECK (total_amount >= 0)  
);
```

Explanation: Each order belongs to one customer. The REFERENCES syntax is a short way to define a foreign key.

5.5 Order Items

```
CREATE TABLE order_items (  
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
order_id INTEGER NOT NULL REFERENCES orders(id),
product_id INTEGER NOT NULL REFERENCES products(id),
quantity INTEGER NOT NULL CHECK (quantity > 0),
unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0)
);
```

Explanation: This table connects orders and products. CHECK ensures quantity is greater than zero.

6. Check Tables in psql

```
\dt
\d products
```

Explanation: \dt lists tables. \d products shows the structure of the products table. These commands are used inside psql.

7. Insert Sample Data

7.1 Categories

```
INSERT INTO categories (name) VALUES
('Breakfast'), ('Main Course'), ('Drinks'), ('Desserts'), ('Salads');
```

Explanation: This inserts five rows with one INSERT statement.

7.2 Products

```
INSERT INTO products (category_id, name, description, price, metadata) VALUES
(1, 'Egg Breakfast', 'Eggs with bread and vegetables', 28.00,
'{"calories":420,"vegetarian":true}'),
(1, 'Pancakes', 'Pancakes with honey', 24.00, '{"calories":510,"vegetarian":true}'),
(2, 'Chicken Burger', 'Grilled chicken burger', 35.00, '{"calories":650,"spicy":false}'),
(2, 'Beef Burger', 'Beef burger with cheese', 42.00, '{"calories":720,"spicy":false}'),
(2, 'Chicken Pasta', 'Creamy chicken pasta', 48.00, '{"calories":780}'),
(3, 'Orange Juice', 'Fresh orange juice', 18.00, '{"size":"300ml"}'),
(3, 'Coffee', 'Hot black coffee', 14.00, '{"size":"250ml"}'),
(3, 'Water', 'Mineral water', 6.00, '{"size":"500ml"}'),
(4, 'Chocolate Cake', 'Chocolate cake slice', 25.00, '{"calories":430}'),
(4, 'Ice Cream', 'Vanilla ice cream', 20.00, '{"calories":250}'),
(5, 'Caesar Salad', 'Chicken Caesar salad', 32.00, '{"calories":390}'),
(5, 'Green Salad', 'Fresh mixed green salad', 22.00, '{"calories":180,"vegetarian":true}');
```

Explanation: The metadata column demonstrates PostgreSQL JSONB. Different products can store different JSON attributes.

7.3 Customers

```
INSERT INTO customers (name, email, phone, city) VALUES
('Ahmed Ali', 'ahmed@example.com', '0501111111', 'Riyadh'),
('Sara Khan', 'sara@example.com', '0502222222', 'Jeddah'),
('Mohammed Saeed', 'mohammed@example.com', '0503333333', 'Dammam'),
('Ali Hassan', 'ali@example.com', '0504444444', 'Riyadh'),
('Fatima Noor', 'fatima@example.com', '0505555555', 'Khobar');
```

Explanation: These sample records are used by later queries.

7.4 Orders and Items

```
INSERT INTO orders (customer_id, status, total_amount) VALUES
(1, 'Completed', 88.00),
(2, 'Pending', 66.00),
(1, 'Confirmed', 50.00);

INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES
(1, 3, 2, 35.00),
(1, 6, 1, 18.00),
(2, 4, 1, 42.00),
(2, 2, 1, 24.00),
(3, 9, 2, 25.00);
```

Explanation: Customer 1 has two orders. Order 1 contains two burgers and one juice.

8. SELECT - Read Data

```
SELECT * FROM products;
```

Explanation: * returns every column.

```
SELECT name, price FROM products;
```

Explanation: This returns only selected columns.

9. WHERE and Comparison

```
SELECT * FROM products WHERE price > 30;
```

Explanation: Returns products priced above 30.

```
SELECT * FROM customers WHERE city = 'Riyadh';
```

Explanation: Text values are placed inside single quotes.

10. AND, OR, IN and BETWEEN

```
SELECT * FROM products
WHERE price BETWEEN 20 AND 40;
```

Explanation: BETWEEN is a simple way to search within a range.

```
SELECT * FROM customers
WHERE city IN ('Riyadh', 'Jeddah');
```

Explanation: IN checks whether a value matches one of several values.

11. LIKE and ILIKE

```
SELECT * FROM products WHERE name LIKE '%Burger%';
```

Explanation: LIKE performs pattern matching.

```
SELECT * FROM products WHERE name ILIKE '%burger%';
```

Explanation: ILIKE is a useful PostgreSQL feature for case-insensitive pattern matching.

12. ORDER BY and LIMIT

```
SELECT name, price
FROM products
ORDER BY price DESC
LIMIT 5;
```

Explanation: DESC sorts highest to lowest. LIMIT returns only five rows.

13. UPDATE

```
UPDATE products
SET price = 38.00
WHERE id = 3
RETURNING id, name, price;
```

Explanation: UPDATE changes data. PostgreSQL RETURNING immediately shows the changed row.

14. DELETE

```
INSERT INTO products (category_id, name, price)
VALUES (3, 'Test Drink', 1.00);

DELETE FROM products
WHERE name = 'Test Drink'
RETURNING id, name;
```

Explanation: RETURNING also works with DELETE. Always use WHERE carefully.

15. JOIN

```
SELECT
  p.id,
  p.name AS product,
  c.name AS category,
  p.price
FROM products p
JOIN categories c ON p.category_id = c.id
ORDER BY c.name, p.name;
```

Explanation: JOIN combines related rows from products and categories. p and c are aliases.

16. Multi-table Order Report

```
SELECT
  o.id AS order_id,
  c.name AS customer,
  p.name AS product,
  oi.quantity,
  oi.unit_price,
```

```
oi.quantity * oi.unit_price AS line_total
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
ORDER BY o.id;
```

Explanation: This combines four tables to produce a practical order report.

17. Aggregate Functions

COUNT

```
SELECT COUNT(*) AS total_products FROM products;
```

Explanation: Counts rows.

AVG

```
SELECT ROUND(AVG(price), 2) AS average_price FROM products;
```

Explanation: Calculates the average and rounds it to two decimals.

MAX

```
SELECT MAX(price) AS highest_price FROM products;
```

Explanation: Returns the highest price.

MIN

```
SELECT MIN(price) AS lowest_price FROM products;
```

Explanation: Returns the lowest price.

SUM

```
SELECT SUM(total_amount) AS completed_sales FROM orders WHERE status='Completed';
```

Explanation: Adds completed order values.

18. GROUP BY and HAVING

```
SELECT
    c.name AS category,
    COUNT(p.id) AS total_products
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name
ORDER BY total_products DESC;
```

Explanation: GROUP BY creates one result per category. LEFT JOIN keeps categories even if they have no products.

```
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id
HAVING COUNT(*) >= 2;
```

Explanation: HAVING filters grouped results. WHERE filters rows before grouping.

19. Subquery

```
SELECT name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

Explanation: The inner query calculates the average. The outer query finds products above that average.

20. CTE - Common Table Expression

```
WITH expensive_products AS (
  SELECT id, name, price
  FROM products
  WHERE price > 30
)
SELECT *
FROM expensive_products
ORDER BY price DESC;
```

Explanation: A CTE creates a temporary named result for one query. It can make complex SQL easier to read.

21. JSONB Examples

```
SELECT name, metadata
FROM products
WHERE metadata @> '{"vegetarian": true}';
```

Explanation: @> checks whether JSONB contains the requested JSON data.

```
SELECT
  name,
  metadata ->> 'calories' AS calories
FROM products
WHERE metadata ? 'calories';
```

Explanation: ->> returns a JSON value as text. The ? operator checks whether a JSONB key exists.

22. Indexes

```
CREATE INDEX idx_products_name ON products(name);
```

Explanation: A normal index can improve searches by product name.

```
CREATE INDEX idx_products_metadata ON products USING GIN(metadata);
```

Explanation: GIN indexes are commonly useful for searching JSONB data.

23. View

```
CREATE VIEW product_details AS
SELECT
    p.id,
    p.name AS product,
    c.name AS category,
    p.price,
    p.is_active
FROM products p
JOIN categories c ON p.category_id = c.id;

SELECT * FROM product_details;
```

Explanation: A view stores a reusable query definition. It does not normally store a separate copy of the result data.

24. Transaction

```
BEGIN;

UPDATE products
SET price = price + 2
WHERE category_id = 3;

COMMIT;
```

Explanation: BEGIN starts a transaction and COMMIT saves its changes.

```
BEGIN;

UPDATE products
SET price = price + 100
WHERE category_id = 3;

ROLLBACK;
```

Explanation: ROLLBACK cancels changes made in the current transaction.

25. PostgreSQL UPSERT

```
INSERT INTO customers (name, email, city)
VALUES ('Ahmed Updated', 'ahmed@example.com', 'Riyadh')
ON CONFLICT (email)
DO UPDATE SET
    name = EXCLUDED.name,
    city = EXCLUDED.city
RETURNING *;
```

Explanation: ON CONFLICT handles a duplicate unique value. Here, an existing customer with the same email is updated instead of causing an error.

End of PostgreSQL Assignment & Notes